Making a Database Think Fast

Five ways to speed up recursive queries — and which ones earned their keep

27 August 2026 · a working note, written to be read on paper

Start from zero: what problem are we solving?

Imagine a database — a big table of simple facts. Each fact is tiny, like "Alice follows Bob" or "road A connects to road B". On their own these facts are boring. The interesting questions are the ones that **chain** facts together:

> **The question that needs chaining**
>
> "Who can Alice reach, following the chain of who-follows-whom, no matter how many hops it takes?" There is no single fact that answers this. The answer is built by following facts into more facts, over and over, until you have found everyone.

A question that keeps applying the same rule to its own results is called **recursive**. And the little language for asking these questions over a database is called **Datalog**. You write a rule once, and the system keeps applying it until nothing new comes out:

```
reachable(A, B) if there is a direct edge A → B.
reachable(A, B) if A → X, and X can reach B.   ← the rule uses itself
```

The second line is the whole trick: `reachable` is defined **in terms of itself**. Run it and it discovers every reachable pair, however far apart.

Why the obvious way is slow

The simplest way to compute this is to keep sweeping over all the facts, deriving new ones, until a sweep produces nothing new. That works — but it redoes enormous amounts of work. Each sweep re-derives everything it already knew, just to find the few genuinely new facts at the frontier.

The standard fix is called **semi-naïve evaluation**, and the idea is simple:

> **The one idea behind semi-naïve**
>
> A fact can only be **new** this sweep if it was built from a fact that was **new last sweep**. So don't re-examine everything each round — only push on the facts that just appeared. Each fact then gets derived roughly once, instead of once per remaining round.

Our system already does this. But profiling it revealed two separate weaknesses, and they call for completely different fixes:

1770 facts computed to answer a 59-fact question

Asking "who is reachable from **one** node" still built the entire web of reachable pairs, then threw almost all of it away.

~1 second for a chain of 60 links

The method was right, but running it in the interpreted language was slow per-fact — the overhead was in the **machinery**, not the **algorithm**.

These two problems map onto two different families of fix. One is about doing **less work** (be smarter about which facts to compute). The other is about doing the **same work faster** (a better engine underneath). The literature offers five techniques across these two families; I prototyped and measured all five.

The five techniques, in plain words

Family A — compute fewer facts (algorithmic)

Axis 1 · Magic Sets — only chase what the question needs. If you ask "who can Alice reach", there is no point computing who **Bob** can reach unless Bob is on a path from Alice. Magic Sets rewrites the rules so the computation is **pulled** by the question: it starts from Alice and only derives facts that could possibly matter. It is a rewrite of the rules themselves — rules go in, smarter rules come out — so it needs no new engine.

Axis 4 · PreM — carry only the best answer as you go. For "shortest route" questions, the naïve rules would compute **every** route, including ones that loop forever around a cycle, and only pick the shortest at the end. PreM ("push the minimum into the recursion") keeps just the best cost found **so far** for each place, discarding worse routes immediately. That both avoids the endless loop and shrinks the work.

Family B — run the same work faster (engine)

Axis 5 · A native engine. The database's storage is already written in Rust (a fast, compiled language). So the natural idea: even though the query is **written** in the high-level language, run the actual grinding — the sweeps, the joins — in Rust, over plain integers instead of boxed language objects.

Axis 2 · Indexed joins. When the engine combines two sets of facts ("for each edge out of X, find where it leads"), the slow way scans every fact each time. An **index** is a lookup table that jumps straight to the matching facts, the way a book's index beats reading every page.

Axis 3 · Incremental maintenance. A database is **alive** — facts get added over time. When one new fact arrives, you shouldn't recompute everything from scratch; you should nudge the existing answer to account for just that change. (This is the same idea as semi-naïve, but applied to **updates** rather than the first computation.)

What I measured

I built a native engine as a **spike** — a throwaway prototype whose only job is to produce honest numbers — and ran every technique against the same inputs, checking each answer matched the trusted slow version exactly.

Axis 5 — native vs interpreted (the biggest surprise)

chain length	interpreted	native	speed-up
20 links	97 ms	3 ms	**32×**
40 links	329 ms	5 ms	**65×**
60 links	1117 ms	8 ms	**139×**

Same algorithm, same answers (verified identical). The only change is where it runs. The speed-up grows because the interpreter's per-fact overhead piles up as the work grows.

32× to 139×, and widening

The native engine is not a little faster — it is a different league, and the gap grows with the size of the problem.

Axis 1 — Magic Sets (compute only what's asked)

On a graph made of 80 separate little chains, asking about **one** chain:

approach	facts computed	time
full computation	8400	25 ms
Magic Sets	**105**	**12 ms**

*Magic Sets touched only the chain the question was about, and **ignored the other 79 entirely**. Its cost stays flat no matter how big the irrelevant part of the graph grows.*

Axis 2 — indexed joins

On a branching graph (where each node leads to several others), the index paid off — 61 ms dropped to 14 ms, a 4× win that grows with size. On a plain chain (each node leads to exactly one other) the index had nothing to accelerate, and correctly showed no benefit. The technique helps exactly when there is fan-out to exploit.

Axis 4 — PreM (shortest paths, even with loops)

On a weighted graph **containing a cycle** — the case that makes the naïve method loop forever — PreM converged cleanly and found the true shortest costs: a 2-hop route of cost 3 correctly beat a tempting direct road of cost 10. The key result is not the speed but that it **finishes at all** where the textbook version cannot.

Axis 3 — incremental (the honest miss)

This one broke even, and I want to be plain about why. My prototype re-loaded the whole existing answer each time before applying the change — so the cheap part (handling the new fact) was drowned by the expensive part (reloading). The technique is sound; my **harness** was wrong. Doing it properly needs the answer to **stay resident** in memory between updates, which is a real design, not a quick spike. Reported as break-even, not dressed up as a win.

The decision — and the mistake I nearly made

Here is where it gets interesting, and where I had to stop and reconsider.

My first instinct was to build the whole engine — rules, recursion, all of it — in Rust. But this project has a hard-won principle, written down after earlier work: **one representation**. Keep a single source of truth. The moment you have "the rules, as the high-level language understands them" **and** "the rules, as a separate Rust engine understands them", you have two things that must agree forever, and a translator to maintain between them. An earlier decision in this same codebase rejected exactly this — a second engine that has to be kept in lock-step is a well-known trap.

> **The reconciliation**
>
> The two **algorithmic** techniques (Magic Sets, PreM) are just **rewrites of the rules** — rules in, better rules out. They are not an engine at all, so they belong in the high-level language, where they compose with everything else the query system can already do. I built them there, and they work.
>
> The **engine** speed-up (native execution) is real and huge — but it should be a **scoped accelerator** for the narrow, mechanical task of grinding a fixpoint over integers, the way the storage layer is already a scoped accelerator for reading bytes. It must never grow into a second brain with its own opinions about what the rules **mean**.

So the final shape is: **meaning and composition live in the high-level language; raw grinding can be borrowed from Rust** — but only as a dumb, fast muscle, never as a second mind. That honours both the measured 139× and the principle that keeps the system coherent.

Where this leaves things

technique	verdict	home
Semi-naïve	baseline, shipped	language
Magic Sets	WORKS 80× fewer facts	language (rewrite)
PreM	WORKS converges on cycles	language (rewrite)
Native engine	WORKS 32–139×	scoped Rust muscle
Indexed joins	WORKS 4× on fan-out	inside the muscle
Incremental	PARTIAL needs resident state	future work

The short version: **semi-naïve made recursion possible; Magic Sets and PreM make it clever; the native engine makes it quick — and the discipline of "one representation" decides which of those live where.** Every number here was measured on the same machine, against the same trusted answers, and the one technique that didn't pay off is reported as such rather than hidden.

All five prototypes and their benchmarks live in the repository under `native/datom_datalog/` (the Rust spike) and `priv/datom/query/` (`rules.bl`, `magic.bl`, `prem.bl`). 401 existing tests still pass — every addition here was purely additive.